UNIT 1 — Database Basics

Que 1. Explain the different types of Database Systems with examples.
Ans:

Types of Database Systems – Complete Answer (10 Marks)

1. Introduction

A **Database System** is a software system that allows us to **store, organize, manage, and retrieve data efficiently**.

- Without a database, data is usually stored in separate files which is **difficult to manage**.
- Database systems **reduce redundancy**, maintain **data integrity**, and provide **fast access** to data.

2. Types of Database Systems

A. Relational Database System (RDBMS)

- **Definition:** Data is stored in **tables** (rows and columns). Relationships between tables are maintained using **keys**.
- **Features:**
  - Structured data storage in tables
  - Uses **SQL (Structured Query Language)** to query data
  - Ensures **data consistency, accuracy, and security**
- **Example:** MySQL, Oracle, Microsoft SQL Server
- **Use Case:**

    - Banking systems (transactions)
    - Online shopping websites (product orders)
    - Student management systems (marks, attendance)

B. NoSQL Database System

- **Definition:** Non-relational databases that store data in **key-value pairs, documents, columns, or graphs**. Suitable for **large and unstructured data**.
- **Features:**
    - Flexible schema – no fixed structure required
    - Highly scalable and fast for large datasets
    - Handles **unstructured and semi-structured data**
- **Example:** MongoDB (Document DB), Redis (Key-Value DB), Cassandra (Column-family DB), Neo4j (Graph DB)
- **Use Case:**
    - Social media apps (Facebook, Instagram)
    - Real-time analytics (stock market, weather apps)
    - Big data applications (IoT, sensor data)

C. Hierarchical Database System

- **Definition:** Data is stored in a **tree-like structure** with **parent and child nodes**.
- **Features:**
    - Efficient for **one-to-many relationships**
    - Quick data retrieval if hierarchy is known
- **Example:** IBM IMS (Information Management System)
- **Use Case:**
    - Airline reservation systems
    - File systems in computers

D. Network Database System

- **Definition:** Data is stored as **records connected by links**, allowing **many-to-many relationships**.
- **Features:**
  - Supports complex relationships
  - More flexible than hierarchical DB
  - Efficient for complex queries
- **Example:** Integrated Data Store (IDS)
- **Use Case:**
  - Telecommunication networks
  - Logistics and supply chain management

E. Flat File Database System

- **Definition:** Data is stored in **simple files**, like text files or spreadsheets.
- **Features:**
  - Easy to use for small amounts of data
  - Simple and does not require complex software
  - Not suitable for large or related datasets
- **Example:** CSV files, Excel sheets
- **Use Case:**
  - Small businesses (billing, inventory)
  - Contact lists
  - Simple lists or records

3. Summary Table

Type	Structure	Example	Use Case
Relational (RDBMS)	Tables (Rows & Columns)	MySQL, Oracle	Banking, E-commerce
NoSQL	Key-Value / Document / Graph / Column	MongoDB, Redis	Social Media, Real-time Apps
Hierarchical	Tree structure	IBM IMS	Airline Reservation

Network	Records with links	IDS	Telecom
Flat File	Simple files	CSV, Excel	Small business, lists

4. Diagram

```
                    TYPES OF DATABASE SYSTEMS
                              |
   ------------------------------------------------
   |           |           |           |           |
RDBMS       NoSQL     Hierarchical   Network    Flat File
(MySQL)  (MongoDB)       (IMS)       (IDS)     (CSV/Excel)
```

(Exam me aap boxes aur arrows draw kar sakte ho for neat presentation)

5. Conclusion

- Database systems help **store data efficiently**, **maintain integrity**, and **reduce redundancy**.
- The **choice of database** depends on:
  - Type of data (structured or unstructured)
  - Size of data
  - Speed and access requirements
  - Complexity of relationships

==========

Que 2. What is Relational Database Design? Explain its structure and advantages.

Ans:

Relational Database Design – 10 Marks Answer

1. Introduction

- **Relational Database Design (RDB Design)** is the process of **organizing data into tables (relations) to reduce redundancy and maintain accuracy**.
- It is used in **Relational Database Management Systems (RDBMS)** like **MySQL, Oracle, SQL Server**.
- It helps in **efficient data storage, retrieval, and management**.

Example:

- School wants to store **student details, courses, and marks**. RDB design ensures that **student info is stored only once**, and relationships are maintained between students and courses.

2. Structure of Relational Database

A relational database stores data in **tables**, where each table represents an **entity** (like Student, Employee, Product).

Key Components:

1. **Tables (Relations):**
    a. A table represents a real-world entity.
    b. Example: `STUDENT`, `COURSE`, `ENROLLMENT`
2. **Rows (Tuples):**
    a. Each row is a single record in the table.
    b. Example: One student's information
3. **Columns (Attributes):**
    a. Each column represents a property of the entity.

    b. Example: `StudentID`, `Name`, `Class`

4. **Primary Key (PK):**
    a. Unique identifier for each row.
    b. Example: `StudentID` in STUDENT table
5. **Foreign Key (FK):**
    a. A field in one table that refers to **primary key in another table** to maintain relationships.
    b. Example: `StudentID` in ENROLLMENT table
6. **Relationships:**
    a. **One-to-One (1:1):** One student has one locker.
    b. **One-to-Many (1:N):** One teacher teaches many students.
    c. **Many-to-Many (M:N):** Students enroll in many courses, each course has many students (handled using ENROLLMENT table).

3. Diagram of Relational Database Structure

```
     STUDENT TABLE
-----------------------------
StudentID | Name  | Class | Age
-----------------------------
   101    | Ravi  | 10A   | 15
   102    | Priya | 10B   | 14

      COURSE TABLE
-----------------------------
CourseID | CourseName | Credits
-----------------------------
   C01   | Maths      | 4
   C02   | Science    | 3

    ENROLLMENT TABLE
-----------------------------
StudentID | CourseID
-----------------------------
   101    | C01
   101    | C02
```

```
102     | C02
```

Explanation:

- `StudentID` is primary key in STUDENT table.
- `CourseID` is primary key in COURSE table.
- ENROLLMENT table handles **many-to-many relationship** using foreign keys.

4. Advantages of Relational Database Design

1. **Reduces Data Redundancy:**
    a. Same data is not repeated in multiple places.
2. **Ensures Data Integrity:**
    a. Relationships between tables maintain **accuracy and consistency**.
3. **Easy Data Retrieval:**
    a. SQL allows **complex queries** to get data quickly.
4. **Flexibility:**
    a. Tables can be added or modified **without affecting other tables**.
5. **Security:**
    a. Access can be controlled at **table or column level**.
6. **Scalability:**
    a. Can handle **small to large applications** like banking, e-commerce.

5. Real-World Examples

1. **School Database:**
    a. Tables: STUDENT, TEACHER, COURSE, ENROLLMENT
    b. Ensures students' marks and course enrollment are organized.
2. **Bank Database:**
    a. Tables: CUSTOMER, ACCOUNT, TRANSACTION
    b. Ensures transaction history is linked to correct customer.
3. **E-commerce Website:**
    a. Tables: PRODUCT, CUSTOMER, ORDER, PAYMENT
    b. Tracks orders, payments, and customer details efficiently.

6. Conclusion

- **Relational Database Design** organizes data **efficiently, reduces redundancy, and maintains relationships**.
- It is the **backbone of modern applications** and helps developers manage **structured data easily**.

==========

Que 3. Explain the Hierarchical Data Model System with a neat diagram.

Ans:

Hierarchical Data Model System

1. Introduction

- **Hierarchical Data Model** is a way of **organizing data in a tree-like structure**.
- It was one of the **earliest database models**, used before relational databases became popular.
- In this model, **data is stored as records (nodes)**, and the **relationship between records** is maintained through **parent-child links**.
- **Key Idea:** Each **parent record can have multiple child records**, but each **child record has only one parent**. This is called a **one-to-many (1:N) relationship**.

Example:

- In a **company**, a **Department** is a parent, and its **Employees** are children.
- This model is suitable for applications where **data has a clear hierarchy**, like organizational structures, file systems, and banking.

2. Structure of Hierarchical Database

The database is organized like a **tree**, where nodes represent **records** and branches represent **relationships**.

Key Components:

1. **Records (Nodes):**
    a. Each record stores information about an entity.
    b. Example: `Employee Record` contains EmployeeID, Name, and Salary.
2. **Parent-Child Relationship:**
    a. **Parent Node:** Main record that can have multiple child records.
    b. **Child Node:** Record that belongs to a parent.
    c. Example: Department (Parent) → Employees (Children)
3. **Tree Structure:**
    a. **Root Node:** Top-most parent record in the hierarchy.
    b. **Leaf Nodes:** Bottom-most child records with no children.
    c. Each child has **exactly one parent**, but one parent can have **many children**.
4. **1:N Relationship:**
    a. One parent can have multiple children.
    b. Example: A department can have many employees, but each employee belongs to only one department.

3. Diagram of Hierarchical Data Model

```
            COMPANY (Root Node)
                   |
        --------------------------
        |                        |
   DEPARTMENT 1              DEPARTMENT 2
        |                        |
   -----------               -----------
   |    |    |                |     |
Emp1  Emp2  Emp3            Emp4   Emp5
```

Explanation:

- `COMPANY` = Root node
- `DEPARTMENT` = Parent nodes
- `EMPLOYEE` = Child nodes
- Shows **1:N relationship** – One department has many employees.

4. Advantages of Hierarchical Data Model

1. **Simple and Easy to Understand:**
    a. Tree-like structure is easy to visualize.
2. **Fast Data Retrieval:**
    a. Data can be accessed quickly if the path from root to child is known.
3. **Maintains Data Integrity:**
    a. Parent-child relationship ensures **accuracy and consistency** of data.
4. **Efficient for 1:N Relationships:**
    a. Perfect for applications with **fixed hierarchical data**, like departments and employees.
5. **Clear Structure:**
    a. Each level of hierarchy clearly shows the relationship.

5. Limitations / Disadvantages

1. **Difficult to Modify:**
    a. Adding new relationships or changing hierarchy is hard.
2. **Data Redundancy:**
    a. Same data may need to be repeated in multiple places.
3. **Limited Relationships:**
    a. Only supports **1:N relationships**; **many-to-many (M:N)** relationships are difficult.
4. **Complex Queries:**
    a. Retrieving data that spans multiple branches can be complicated.

6. Real-Life Examples

1. **Organization Database:**
    a. Company → Departments → Employees
    b. Each department has many employees; each employee belongs to one department.
2. **University Database:**
    a. University → Colleges → Courses → Students
3. **Bank Database:**
    a. Bank → Branches → Accounts → Customers
4. **File System:**
    a. Root Directory → Folders → Subfolders → Files

7. Conclusion

- The **Hierarchical Data Model** organizes data in a **tree-like structure**, making it **easy to navigate and maintain one-to-many relationships**.
- It is **fast, simple, and maintains data integrity**, but **less flexible** than modern relational or NoSQL databases.
- Best suited for applications where **hierarchy is fixed**, such as **banking, airline reservations, company management, and file systems**.